



GE VERNOVA

ADMS

16.25.1

Modeling Reference Guide

Copyright

Copyright 2026, GE Vernova and/or its affiliates ("GE Vernova"). All Rights Reserved.

GE and the GE Monogram are trademarks of General Electric Company used under trademark license.

This document is the confidential and proprietary information of GE Vernova and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE Vernova.

Change History

Date	Change Description
May 2, 2023	ADMS 3.16: • Updated In-Memory Data Storage. • Editorial and formatting updates.
Oct 20, 2023	ADMS 16.1.0: • • The product version numbering has changed. The prefix '3' is no longer included in the version numbering for ADMS.
Apr 1, 2024	ADMS 16.7.0: • Updated Adding Custom Fields in DNOM. • Updated document format and styling.

Contents

1. Introduction	7
1.1. Terminology	7
2. External Data Model Format	8
2.1. Modeling Reference Tables Excel Workbook	8
2.1.1. About the “Attribute Type”	8
2.1.2. Overview of Input Files	8
2.1.3. Station Input Table	9
2.1.4. Measurement Types Table	12
2.1.5. Globals Input Table	13
2.1.6. Defaults CSV Input Table	14
2.1.7. Query Objects Table	15
2.1.8. Query Object Mapping Table	15
2.1.9. Transformer Enumerations Table	15
2.1.10. Transformer Configurations Table	16
2.1.11. Schedule Point Validation Table	16
2.1.12. Conductor Model Mandatory Fields Table	17
2.2. Transformer Types, Configurations, and Models	17
2.2.1. Transformer Type Terminology	18
2.2.1.1. Transformer Type	18
2.2.1.2. Transformer Configuration Type	19
2.2.2. How CONVERTER Determines Types	20
2.2.2.1. How CONVERTER Determines the Transformer Type (DMS Type)	21
2.2.2.2. How CONVERTER Determines the Transformer Configuration Type (tftype.dat Set)	21
2.3. Input Conventions and Guidance	22
2.3.1. Feeder Colors	22
2.4. DNOMConverter XML Schema	22
2.4.1. Station Name Constraints	23
2.4.2. Metric and Imperial Units	23
2.4.3. Voltage Base	23
2.5. Global File Definition	24
2.5.1. Global File - Guidance and Examples	25
2.5.1.1. Modeling a Set of Seasons	25
2.5.1.2. Modeling Day Types	25
3. Internal Data Model Format	27
3.1. DNOM Description	27
3.1.1. Adding Custom Fields in DNOM	28
3.1.2. Commissioning/Decommissioning Equipment in DNOM	29
3.2. DNOM Query Basics	31
3.2.1. Creating DNOM Queries Graphically	33

3.2.2. Querying the List of DNOM Objects and Fields	33
3.3. Virtual Object Queries	36
3.4. DNOM Group Queries	36
3.5. DNOM Component Queries	36
3.5.1. Substation	37
3.5.1.1. Input Attributes	37
3.5.1.2. Output Attributes	37
3.5.2. Feeder	37
3.5.2.1. Input Attributes	37
3.5.2.2. Output Attributes	37
3.5.2.3. Virtual Attributes	37
3.5.3. Switch	38
3.5.3.1. Input Attributes	38
3.5.3.2. Output Attributes	38
3.5.3.3. Virtual Attributes	38
3.5.4. Line	39
3.5.4.1. Input Attributes	39
3.5.4.2. Output Attributes	39
3.5.4.3. Virtual Attributes	39
3.5.5. Transformer	40
3.5.5.1. Input Attributes	40
3.5.5.2. Output Attributes	40
3.5.5.3. Virtual Attributes	40
3.5.6. Distribution Transformer	41
3.5.6.1. Input Attributes	41
3.5.6.2. Output Attributes	41
3.5.6.3. Virtual Attributes	41
3.5.7. Load	41
3.5.7.1. Input Attributes	41
3.5.7.2. Output Attributes	42
3.5.7.3. Virtual Attributes	42
3.5.8. Shunt Impedance	42
3.5.8.1. Input Attributes	42
3.5.8.2. Output Attributes	42
3.5.8.3. Virtual Attributes	42
3.5.9. Series Reactor	43
3.5.9.1. Input Attributes	43
3.5.9.2. Output Attributes	43
3.5.9.3. Virtual Attributes	43
3.5.10. Generator	44
3.5.10.1. Input Attributes	44
3.5.10.2. Output Attributes	44

3.5.10.3. Virtual Attributes	44
3.5.11. Energy Resource	44
3.5.11.1. Input Attributes	44
3.5.11.2. Output Attributes	45
3.5.11.3. Virtual Attributes	45
3.5.12. Ground	45
3.5.12.1. Input Attributes	45
3.5.12.2. Virtual Attributes	45
3.5.13. Switchable Sections	46
3.5.13.1. Input Attributes	46
3.5.13.2. Output Attributes	46
3.5.14. Cut	46
3.5.14.1. Input Attributes	46
3.5.14.2. Output Attributes	46
3.5.14.3. Virtual Attributes	46
3.5.15. Temporary Jumper	47
3.5.15.1. Input Attributes	47
3.5.15.2. Output Attributes	47
3.5.15.3. Virtual Attributes	47
3.5.16. Composite Switch	48
3.5.16.1. Input Attributes	48
3.5.16.2. Output Attributes	48
3.5.16.3. Virtual Attributes	48
3.5.17. Bus	48
3.5.17.1. Input Attributes	48
3.5.17.2. Output Attributes	49
3.5.17.3. Virtual Attributes	49
3.5.18. Transformer Bank	49
3.5.18.1. Input Attributes	49
3.5.18.2. Virtual Attributes	49
3.5.19. Time Overcurrent Relay	50
3.5.19.1. Input Attributes	50
3.5.20. Fault Indicator	50
3.5.20.1. Input Attributes	50
3.5.20.2. Virtual Attributes	50
3.5.21. Sensor Location	50
3.5.21.1. Input Attributes	50
3.5.22. Vault	50
3.5.22.1. Input Attributes	50
3.5.22.2. Virtual Attributes	51
3.5.23. Switch Cabinet	51
3.5.23.1. Input Attributes	51

3.5.23.2. Virtual Attributes	51
3.5.24. Duct Bank	51
3.5.24.1. Input Attributes	51
3.5.24.2. Virtual Attributes	52
3.5.25. Manhole	52
3.5.25.1. Input Attributes	52
3.5.25.2. Virtual Attributes	52
3.5.26. Splice Box	52
3.5.26.1. Input Attributes	52
3.5.26.2. Virtual Attributes	52
3.5.27. Trouble Ticket	53
3.5.27.1. Input Attributes	53
3.5.28. Tags	53
3.6. Global Object Queries	53
3.6.1. Construction Model	53
3.6.2. Conductor Model	53
3.6.3. Load Category	54
3.6.4. Schedule	54
3.6.5. Schedule Point	54
3.6.6. Season	54
3.6.7. Day Type	54
3.6.8. Transformer Model	55
3.6.9. Tap Changer Model	55
3.6.10. Switch Model	55
3.6.11. Shunt Impedance Model	55
3.6.12. Series Reactor Model	55
3.6.13. Generator Model	55
3.6.14. Energy Resource Model	55
3.6.15. Energy Resource Program Definition	56
3.6.16. Composite Switch Model	56
3.6.17. Optimization Station Group	56
3.6.18. Enumeration Table	56
4. In-Memory Data Storage	57
5. Viewing Station Model Data	58

1. Introduction

This guide is divided into two major sections: the external data description and the internal data description.

The external data description provides definitions of the records and fields that are required for describing the complete station facilities and topology/connectivity data. This external data is read into the converter application, which validates the model and generates binary station model files that are used by the ADMS server and client.

After the model files are loaded into ADMS, specific query syntax is used to access the internal data stored in the memory-resident object model called the Distribution Network Object Model (DNOM).

1.1. Terminology

DNOM: Distribution Network Object Model

DPF: Distribution Power Flow

PRV: Protection Validation

FL: Fault Location

UI: User Interface

For more terms, acronyms, and their definitions refer to the *ADMS Glossary of Terms*.

2. External Data Model Format

This chapter describes the objects and properties available for modeling the distribution system. They are used for the "inside the fence" equipment, lines, and measurements (station internals), and the "outside the fence" equipment, lines, and measurements (station externals).

2.1. Modeling Reference Tables Excel Workbook

The *ADMS_Modeling Reference Tables* Excel workbook (`distribution_modeling_reference_tables.xlsx`) accompanies this reference guide and provides the necessary details for querying and inputting electrical models.

The Excel workbook is compatible with Microsoft Excel 2007 and later. You can use Excel's table features such as sorting and filtering to view the subset of rows you are interested in.

Cells that contain the value "#N/A" indicate that the value is not applicable. However, empty cells might indicate incomplete information.

2.1.1. About the "Attribute Type"

The "Attribute Type" column in some worksheets describes the format of the "Queryable Field Name" column. The format of the "Queryable Field Name" column is always character strings, even if the result is actually an integer number (for example, "123"). This column accepts the following formats:

- Integer: In the format "1234"
- Float: In the format "1234.56"
- Boolean: One of the two values: "TRUE" or "FALSE"
- String: Descriptive data, like "Switch Closed"

2.1.2. Overview of Input Files

The content and use of the *ADMS Modeling Reference Tables* is described in the subsequent sections. Each worksheet contains one table, which can be filtered or sorted using standard Excel features.

As described in the *ADMS Modeling Overview and Converter User Guide*, there are multiple modeling inputs and outputs from the system. The table below describes the purpose of each Excel worksheet in the *ADMS Modeling Reference Tables*.

Table 1. ADMS Modeling Reference Tables Worksheets

Worksheet	Input	Editing/Use	Output
Station Input Table	Station.xml or Station_SUB.csv (station externals) Station_internals.xml	Substation Editor Property Names. Query Helper or Field Map Names.	Query Helper or Field Map Names
Measurement Types Table	Describes the measurement types, which can be mapped to SCADA ANALOGs, POINTs, and CTRLs.		
Globals Input Table	globals.csv global_internals.xml	Substation Editor Column Name. Query Helper or Field Map Name.	Query Helper or Field Map Name
Defaults CSV Input Table	DMS_Defaults.csv	Default parameters of DNOM objects.	
Query Objects Table		Lists all queryable attributes, including DNOM objects, global objects, and group object queryable attributes.	Query Helper or Field Map
Query Object Mapping Table		Describes the mappings between group objects (DNOM group objects and virtual objects) and the actual queryable objects.	
Transformer Enumerations Table		This is a reference table of system enumerations for transformers.	
Transformer Configurations Table		List of Transformer Configuration Types.	
Schedule Point Validation Table		Describes the schedule point validation fields.	
Conductor Model Mandatory Fields Table	globals.csv	Describes the conductor model mandatory fields.	

2.1.3. Station Input Table

The station input consists of station internals, station externals, global models, and default files:

- The station data is divided into two parts: the station internal data and the station external data. The station internal data is used to describe the devices and connectivity within the substation fence. The station external data is used to describe the devices and connectivity outside the substation fence.

Station external data may be provided either in XML format (with the file name `station.XML`) or in CSV format (with the file name `station_SUB.CSV`).

Station internal data is provided in XML format in files named `station_INTERNALS.XML` and `station_PLACEMENTS_view.XML`.

- In addition to the station data, a global data file named `global.csv` defines models and objects that are common to multiple records in the station internal and station external data files.

Models defined in the Substation Editor are provided in the `global_internals.xml` file. Optionally, a default file named `DMS_Defaults.csv` can be used to set defaults for station facility information.

- The `MapperTable.xml` configuration file must be available to the converter. Do not change this file. This file maps the equipment names and property names among the CSV column headers, the XML tags, the DNOM Query Helper name, and the CIM name. The Station Input table contains the following columns:
- **Sort:** The Sort identifier enables grouping of related properties within a given equipment type.
- **Type:** Except for the “UI Container” and “Value Placement” types, the following are equipment types. The UI Container is a collection of contiguous equipment, which is grouped for display purposes. Value placements are used to link measurements and arrows visible on the geographic view to DNOM objects.

The Type column has the following values:

- Substation
- Feeder
- Bus
- Loop End Label
- Generator
- Energy Resource
- Line
- Composite Switch
- Switch
- Fuse
- Breaker
- Recloser
- Sectionalizer
- Jumper
- Load
- Series Reactor
- Shunt Impedance
- Ground
- Transformer Bank
- Transformer
- Time Overcurrent Relay
- Fault Detector

- Switch Cabinet
- Value Placement
- Vault
- UI Container
- Static VAR Compensator
- Manhole
- Duct Bank
- Splice Box
- **Field Name (XML):** Name of the field in the station externals XML file or the `station_INTERNALS.XML` file.
- **Field Name (CSV):** Name of the field in the station externals CSV file.
- **Authoritative Source:** If the same property is provided, directly or indirectly, for the same piece of equipment in the station externals file and the station internals file, this column indicates which value is used.
- **Description:** Attribute description or comment.
- **Attribute Type:** Format for the attribute (Boolean, string, integer, or floating point value).
- **Imperial Units**
- **Metric Units**
- **Scope:** Indicates the converter validation scope level, either OPS (which means that the field is required for the operations model scope) or DPF (which means that the field is required for the power flow-based applications).
- **Default:** Indicates the converter set default value, or if the value can optionally be set in the `DMS_Defaults.csv` file.
- **Possible Values or Reference:** Indicates possible values (such as for phasing information) or references to other objects in the file or `global.csv` file.

This field also indicates if the attribute is required or optional.

- **Required?:** Indicates whether a value must be given in the input files.
- **Low Value Imperial Units:** For certain numeric fields, the low value that is used for validation (such as line length).
- **High Value Imperial Units:** For certain numeric fields, the high value that is used for validation (such as line length).
- **Low Value Metric Units:** For certain numeric fields, the low value that is used for validation (such as line length).
- **High Value Metric Units:** For certain numeric fields, the high value that is used for validation (such as line length).
- **Queryable Field Name:** Describe the name of the SQL query statement as described in section [DNOM Query Basics](#).

2.1.4. Measurement Types Table

This section describes the measurement types, which can be mapped to SCADA ANALOGs, POINTs, and CTRLs. It contains the following columns:

- Measurement Type

The DNOM label. This string appears as the value for the <MeasType> property of the desired <Status>, <Analog>, or <Ctrl>.

- Valid Measurement Group

The name of the DNOM object for which this is a valid <MeasType>. It is also the prefix of the valid <MeasGroup>:

Status: StatusMeasurementGroup

Analog: AnalogMeasurementGroup

Ctrl: CtrlMeasurementGroup

- Enumeration: For GE use only.

- Source

SCADA: SCADA supplies the value to DMS

DMS: DMS supplies the value to SCADA

Deprecated: Check with your project engineer before using this measurement type. As migration plans and tools are announced, support will be scheduled to expire.

- Comment if Source is Deprecated

The measurement type to use instead of this one. Check with your project engineer first.

- Associated Device Type(s)

The DNOM equipment for which this measurement type is meaningful.

- Associated Node(s)

The proper node at which to locate the measurement.

- SWO Function

The Switching Order Function to use in a step for testing this measurement.

- DMS Usage

How this measurement affects DMS behavior.

- Valid Phases

Not all measurements are supported for all combinations of phases. The supported ones are listed here.

- SDIS=0 meaning

This column shows what a zero SDIS means after all modeled and configured inversions have been performed. For three and four state statuses, it shows a "table" of meanings for all combinations of SDIS and XDIS.

- SDIS=1 meaning

This column shows what a set SDIS means after all modeled and configured inversions have been performed.

2.1.5. Globals Input Table

This table describes the global file input fields and contains the following columns:

- Sort
- Model
- Substation Editor Column Name
- Global Internals XML Field Name
- Corresponding Global CSV Field Name
- Attribute Type: Format for the attribute (Boolean, string, integer, or floating point value).
- Imperial Units
- Metric Units
- Description: Attribute description or comment.
- Scope: Indicates the converter validation scope level, either OPS (which means that the field is required for the operations model scope) or DPF (which means that the field is required for the power flow-based applications).
- Possible Values or Reference: Indicates possible values (such as for phasing information) or references to other objects in the file or global.csv file. This field also indicates if the attribute is required or optional.
- Required?: Indicates whether a value must be given in the input files.
- Low Value Imperial Units: For certain numeric fields, the low value that is used for validation (such as line length).
- High Value Imperial Units: For certain numeric fields, the high value that is used for validation (such as line length).
- Low Value Metric Units: For certain numeric fields, the low value that is used for validation (such as line length).
- High Value Metric Units: For certain numeric fields, the high value that is used for validation (such as line length).
- Queryable Field Name: Describes the name of the SQL query statement as described in section [DNOM Query Basics](#).

For more information, see section [Global File Definition](#).

2.1.6. Defaults CSV Input Table

This table describes the default input fields and contains the following columns:

- Sort
- Type: The Type column has the following values:
 - Default Generator
 - Default Bus
 - Default Line
 - Default Breaker
 - Default Recloser
 - Default Sectionalizer
 - Default Load
 - Default Shunt Impedance
 - Default Ground
 - Default Time Overcurrent Relay
 - Default Transformer
- CSV Field Name
- Description: Attribute description or comment.
- Attribute Type: Format for the attribute (Boolean, string, integer, or floating point value).
- Units (Imperial)
- Units (Metric)
- Scope: Indicates the converter validation scope level, either OPS (which means that the field is required for the operations model scope) or DPF (the field is required for the power flow-based applications).
- Low Value Imperial Units: For certain numeric fields, the low value that is used for validation (such as line length).
- High Value Imperial Units: For certain numeric fields, the high value that is used for validation (such as line length).
- Low Value Metric Units: For certain numeric fields, the low value that is used for validation (such as line length).
- High Value Metric Units: For certain numeric fields, the high value that is used for validation (such as line length).

2.1.7. Query Objects Table

This table describes queryable output attributes including components, placements, and pointer objects. This table contains the following columns:

- Sort: This is used to sort and present the table in a reasonable order. You can sort or filter by any column.
- Query Object Type (DNOM Component, DNOM Component Placement, DNOM Group Object, Global Object, Virtual Object, or DNAF Object)
- Record Description
- Record Name
- Input or Output
- Query Field Name
- Description: Attribute description or comment.
- Attribute Type (String can be cast to this type): Format for the attribute (Boolean, string, integer, or floating point value).
- Scope
- Unit (Metric)
- Unit (Imperial)
- Related Field: The parent record for a pointer or group query field.

2.1.8. Query Object Mapping Table

The Query Object Mapping table describes the mapping of group objects (DNOM group objects and virtual objects) to real queryable object records. This table contains the following columns:

- Object Type (Virtual Object or DNOM Group Object)
- Object Description
- Object Name
- Mapped Real Queryable Object Record Name
- Type

2.1.9. Transformer Enumerations Table

This table describes the system enumerations related to transformer types and contains the following columns:

- Enumeration
- Value

- Literal
- XML or SUB.CSV Property Name
- Comments

2.1.10. Transformer Configurations Table

This table describes the list of system Transformer Configuration Types and their attributes. It contains the following columns:

- Transformer Configuration Type
- Description
- TfType Name
- Phasing
- Primary Winding
- Secondary Winding
- Tertiary Winding
- Secondary Lagging Primary Phase (Deg)
- Tertiary Lagging Primary Phase (Deg)
- Secondary Center Tap Grounded
- Tertiary Center Tap Grounded
- Number of Transformers
- Revised Phasing (Primary-Secondary)
- Number of Banks
- Comments

2.1.11. Schedule Point Validation Table

This table describes the schedule point validation fields and contains the following columns:

- Schedule Name
- Schedule Type
- Description
- Axis
- Axis Description
- Unit
- Min Value
-

2.1.12. Conductor Model Mandatory Fields Table

This table describes the conductor model mandatory fields and contains the following columns:

- **Conductor Type:** Conductor type, such as bare wire, secondary cable, or single phase tape shield.
- **DIAMETER_CONDMOD:** Conductor model diameter.
- **DIAMNEUTRAL_CONDMOD:** Conductor model neutral wire diameter.
- **DIAMSHIELD_CONDMOD:** Conductor model shield diameter.
- **GMRNEUT_CONDMOD:** Conductor model neutral wire Geometric Mean Radius (GMR).
- **GMRPHASE_CONDMOD:** Conductor model phase GMR.
- **INSULATION_CONDMOD:** Conductor model insulation.
- **NEUTDIST_CONDMOD:** Conductor model neutral wire distance.
- **NUMSTRANDS_CONDMOD:** Conductor model strand number.
- **PHDIST_CONDMOD:** Conductor model phase distance.
- **RNEUTRAL_CONDMOD:** Conductor model neutral wire radius.
- **RPHASE_CONDMOD:** Conductor model phase radius.
- **SHIELDTHICK_CONDMOD:** Conductor model shield thickness.
- **R1_CONDMOD:** Conductor model positive sequence resistance.
- **X1_CONDMOD:** Conductor model positive sequence reactance.
- **R0_CONDMOD:** Conductor model zero sequence resistance.
- **X0_CONDMOD:** Conductor model zero sequence reactance.

2.2. Transformer Types, Configurations, and Models

This section explains the uses of, and the relationships among, the following objects and properties:

Input File	Object	Properties
GLOBAL.CSV	Transformer	TFMOD_TF
GLOBAL.CSV	Transformer_Model	NAME_TFMOD, TYPE_TFMOD, TFTYPE_TFMOD
GLOBAL_INTERNALS.XML	<Transformer_Model>	<Name>, <Type>, <TfType>
station_SUB.CSV	Transformer	NAME_TFMOD
station.XML	<Transformer>	<TransformerModel>, <CnsType>

Input File	Object	Properties
station_INTERNALS.XML	<Transformer>	<TransformerModel>, <TransType>

- A given transformer model defines a number of properties aside from those discussed in this section. They are defined on the Station Input tab of the *ADMS Modeling Reference Tables*. Within this section, they are simply referred to as "the other model values".
- Both the GLOBAL.CSV file and the GLOBAL_INTERNALS.XML file may define transformer models and their properties. If a model is defined in both GLOBAL.CSV file and in GLOBAL_INTERNALS.XML, that is:

```
<Transformer_Model><Name> = NAME_TFMOD
```

Note that CONVERTER uses the model values from GLOBAL_INTERNALS.XML. This includes:

```
<Transformer_Model><Type> = TYPE_TFMOD
<Transformer_Model><TfType> = TFTYPE_TFMOD
```

as well as the other model values. CONVERTER stores the final values in the CIMDnom.Transformer_Model object.

i Note

The final CIMDnom.Transformer_Model is used for all transformers associated with that model name, regardless of whether the transformers are inside or outside the substation fence.

2.2.1. Transformer Type Terminology

The term "Transformer Type" is used in at least two different ways in the code and in older documentation. This document distinguishes Transformer Type and Transformer Configuration Type.

2.2.1.1. Transformer Type

Transformer Type is sometimes referred to as the "DMS type" of the transformer and is the CIMDnom.Transformer_Model.Type. It is used as an index into the transformer type enumeration.

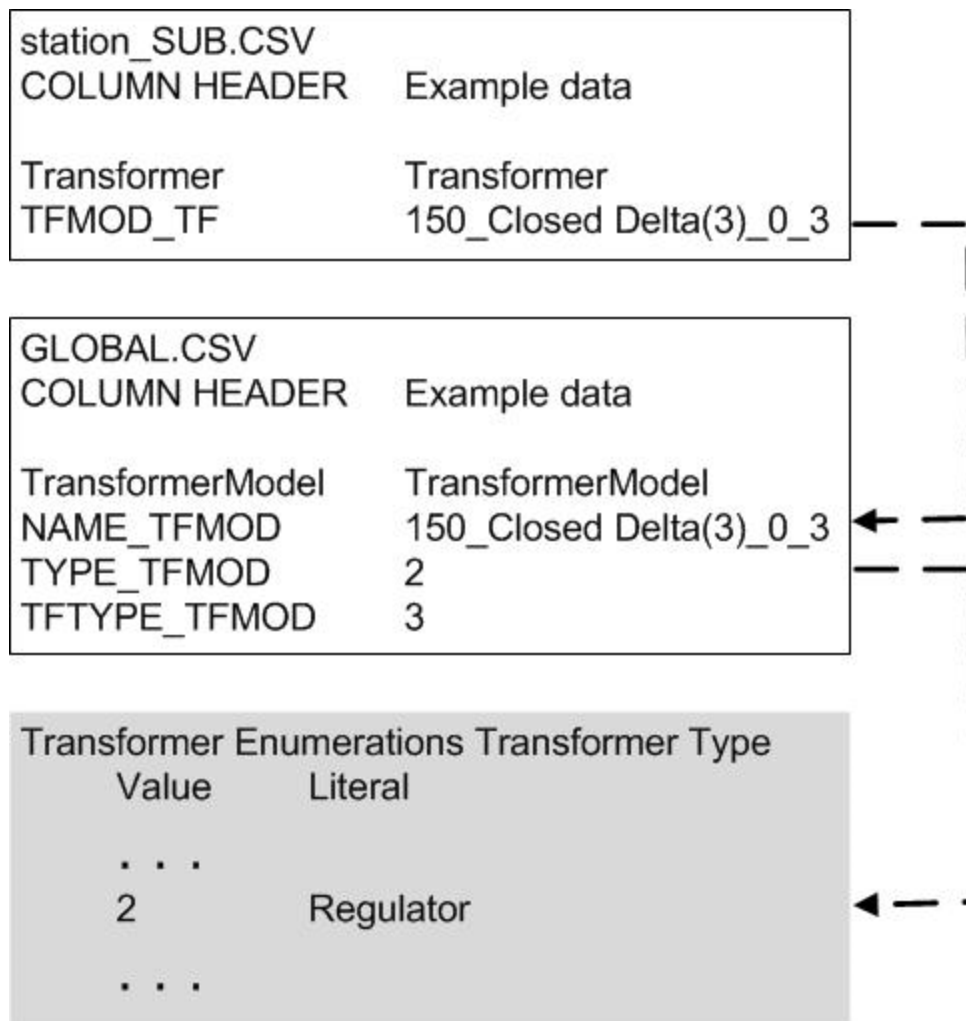


Figure 1. Linking the Modeled Transformer Type to its Enumeration

The supported transformer types are listed on the Transformer Enumerations tab of the *ADMS Modeling Reference Tables*. New transformer types cannot be added without code changes.

2.2.1.2. Transformer Configuration Type

Transformer Configuration Type is determined by the combination of winding types and other properties of a transformer. It is used to find the appropriate parameter set in *tftypes.dat*.

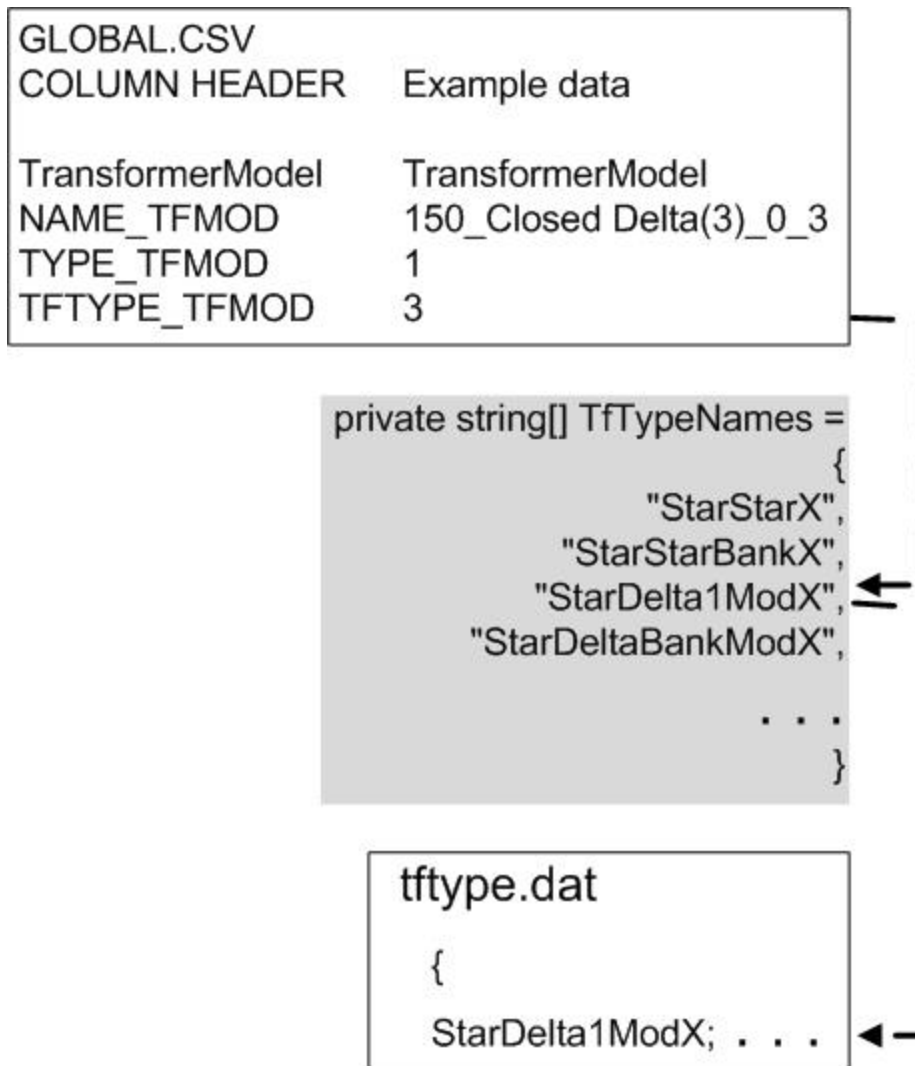


Figure 2. Linking the Transformer Configuration Type to tftype.dat

The supported transformer configuration types (TfTypeNames) are listed on the Transformer Configurations tab of the *ADMS Modeling Reference Tables*. New transformer configuration types cannot be added without code changes.

2.2.2. How CONVERTER Determines Types

A value may come from a station_SUB.CSV file, one of the GLOBAL files, a station_INTERNALS.XML file, or a station.XML file. Which term is used depends upon context. The following terms are equivalent:

TFMOD_TF	<Transformer><TransformerModel>
NAME_TFMOD	<Transformer_Model><Name>
TYPE_TFMOD	<Transformer_Model><Type>
TFTYPE_TFMOD	<Transformer_Model><TfType>

Note

CONVERTER's behavior is different depending upon whether the transformer is defined in a station_SUB.CSV file, a station_INTERNALS.XML file, or a station.XML file.

2.2.2.1. How CONVERTER Determines the Transformer Type (DMS Type)

When TFMOD_TF or <Transformer><TransformerModel> matches some CIMDnom.Transformer_Model.Name, the Transformer DMS type is taken from the model:

`CIMDnom.Transformer.TransType = CIMDnom.Transformer_Model.Type`

When there is no matching transformer model:

- If the input file = station_SUB.CSV or station.XML, CONVERTER declares an error.
- If the input file = station_INTERNALS.XML,

`CIMDnom.Transformer.TransType = <Transformer><Type>`

The same is true for other model values.

2.2.2.2. How CONVERTER Determines the Transformer Configuration Type (tftype.dat Set)

- If the input file = station_SUB.CSV or station.XML, an associated transformer model must be found or CONVERTER declares an error.
- If the input file = station_INTERNALS.XML, and <Transformer><TransformerModel> is not specified, CONVERTER determines the appropriate transformer configuration type from the other properties of <Transformer>, such as the number and type of windings.^[1]

The rest of this section assumes that TFMOD_TF or <Transformer><TransformerModel> matches some CIMDnom.Transformer_Model.Name, and that except in one special case, the CIMDnom.Transformer_Model.TfType is a valid index into the TfTypeNames array.

- If CIMDnom.Transformer_Model.TfType = -1, and input file = station.XML, then `CIMDnom.Transformer.TfType = <Transformer><CnsType>`

Do not use this method of modeling the Transformer Configuration Type. This method will disappear in a future release.

For all input files, if the Transformer has validated this far, CIMDnom.Transformer.TfType is used as an index into the hardcoded array of TfTypeNames to find the tftype.dat name of its parameter set (see Figure "Linking the Transformer Configuration Type to tftype.dat").

2.3. Input Conventions and Guidance

2.3.1. Feeder Colors

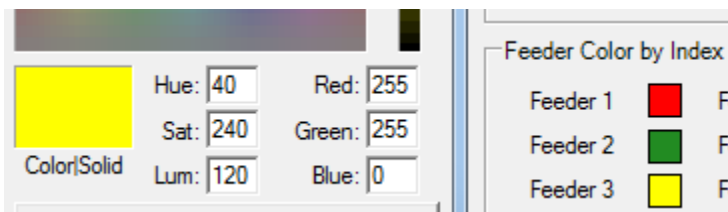
A feeder color is the color used for drawing lines on ADMS graphic displays and on Scada onelines. The feeder heads are drawn "inside the fence" and appear in the ADMS station internals and SCADA substation onelines. The feeder lines are drawn "outside the fence" on the RT, ST, and SIM displays.

"Outside the fence", the feeder color is set using the Viewer Configuration Editor on the Lines tab. In the "Feeder Color by Index" group, each feeder color is associated with a number between one and 24, inclusive. The appropriate feeder color number is specified in the station externals file in the <Feeder><DefColor> property.

"Inside the fence", the feeder color is set by the FG Shared GAB, which is attached to the feeder placement in the Substation Editor. The appropriate feeder color number is part of the GAB name and must be in the format "Fw*COLOR\d{1,2}" where an initial letter "F" is followed by any number (including zero) of alphabetic characters and underscores, followed by the literal "COLOR", followed by one or two digits forming a number between 1 and 24, inclusive. For example, "FEEDERCOLOR2", "FEEDERCOLOR15", "FDR_COLOR6", and "FCOLOR21" are all valid names for Feeder Color GABs.

To ensure that a given feeder has the same color inside and outside the fence, the feeder color number must be associated with the same RGB value both in the Substation Editor and in the Viewer Configuration Editor.

For example, if the Viewer Configuration Editor Lines tab has Feeder Color 3 set to yellow (RGB = 255,255,0):



The matching FEEDERCOLOR3 in the Substation Editor ESET must have its foreground set to the same yellow value:

```
gab "FEEDERCOLOR3"  
(  
  foreground("255,255,0")  
);
```

2.4. DNOMConverter XML Schema

An XML schema file, DNOMConverter.xsd, is provided as the syntactical specification of the XML input files to the Converter (station.xml, station_internals.xml, and global_internals.xml).

When you have a choice of using CSV or XML formatted input files, XML is generally preferred because you can verify syntax conformance against the schema prior to using it in the ADMS tools.

Contact your GE project engineer for advice about input file formats and XML schema compliance.

2.4.1. Station Name Constraints

Station Names must not contain the following characters:

- * Asterisk
- : Colon
- , Comma
- . Dot or Period
- Dash
- < Less Than
- > Greater Than
- " Double Quote
- / Forward Slash
- \ Backslash
- | Vertical Bar or Pipe
- ? Question Mark

2.4.2. Metric and Imperial Units

ADMS supports both metric and imperial units as input and output, which is specified in a Converter configuration flag.

2.4.3. Voltage Base

The Voltage Base input is configurable. The various model elements have an input property for specifying the voltage base, if per unit is not used.

The output voltages can be presented on a user-configurable voltage base (voltage values can be configured through the DNAF configuration application).

2.5. Global File Definition

In addition to the station file(s), there is a global resources file named “global.csv”. The global file mostly contains power flow application data such as conductor and transformer impedances.

The global data file also contains the composite switch models. A composite switch model provides the list of valid states for multi-state switching devices.

The global_internals.xml file has the same objects as the global.csv file, but the format is XML. The tables for the objects contained in the global_internals.xml file are given at the end of this section.

Each field in the global.csv file is a Boolean, a string, an integer, or a floating point number.

- Construction Model
- Conductor Model
- Load Category
- Schedule
- Schedule Point
- Season: See section [Modeling a Set of Seasons](#).
- Day Type: See section [Modeling Day Types](#).
- Transformer Model
- Tap Changer Model
- Switch Model
- Shunt Impedance Model
- Series Reactor Model
- Generator Model
- Composite Switch Model
- Enumeration Table
- VVC Station Group
- Station Name: See section [Station Name Constraints](#).
- Measurement Accuracy Group
- Measurement Accuracy
- Area Division
- Area Power Factor
- SVC Model: Not Supported
- Switch_Model: Global Internals Only (XML Format)
- Series_Reactor_Model: Global Internals Only (XML Format)
- Transformer Model: Global Internals Only (XML Format)

2.5.1. Global File - Guidance and Examples

2.5.1.1. Modeling a Set of Seasons

The following rules must be considered before modeling a set of seasons:

- The set of seasons used for conductor ratings may have overlapping date ranges, as shown in the example below.
- For the conductor model, the limit seasons need to cover an entire year (based on months), otherwise converter issues a warning.
- The month and day for the start and end dates cannot be the same.
- There must be at least one season defined in the global data.

NAME_SEASON	NUMDATERANGE_SEASON	STARTDATE1_SEASON	ENDDATE1_SEASON	STARTDATE2_SEASON	ENDDATE2_SEASON
Default	2	01-Jan	31-May	01-Oct	31-Dec
FALL	1	20-Sep	20-Dec		
WINTER	1	01-Dec	31-May		
SPRING	1	20-Mar	19-Jun		
SUMMER	1	01-Jun	30-Sep		

- The set of seasons used for load categories may not have overlapping date ranges, as shown in the example below.

NAME_SEASON	NUMDATERANGE_SEASON	STARTDATE1_SEASON	ENDDATE1_SEASON	STARTDATE2_SEASON	ENDDATE2_SEASON
default	2	01-Jan	31-May	01-Oct	31-Dec
SUMMER	1	01-Jun	30-Sep		

2.5.1.2. Modeling Day Types

The following rules must be considered before modeling day types:

- Holidays receive special handling and are not considered to overlap with other day types.
- Other day types may not have overlapping ranges.
- There must be at least one day type defined in the global data.

The example below illustrates a valid set of day types.

NAME_DAYTYPE	NUMDAYOFWEEK_DAYTYPE	DAYOFWEEK_DAYTYPE	NUMCALENDARDAY_DAYTYPE	CALENDARDAY_DAYTYPE
Holiday1	0		8	1-Jan-11, 17-Jan-11, 30-May-11, 4-Jul-11

NAME_DAYTYPE	NUMDAYOFWEEK_ DAYTYPE	DAYOFWEEK_ DAYTYPE	NUMCALENDARDAY_ _DAYTYPE	CALENDARDAY_ DAYTYPE
Weekday	5	1,2,3,4,5	0	
Weekend	2	0,6	0	

When a "NUMxxxxxx" column is non-zero, the next column header should be regarded as repeated for the indicated number of entries. In the above example, the "Weekday" row has five "DAYOFWEEK_DAYTYPE" values.

[1] Details of the algorithm can be provided upon request, but it is usually easier to define and reference a <Transformer_Model>.

3. Internal Data Model Format

This chapter provides the information that is necessary to query the GE ADMS server, including the query syntax and a description of the currently available queryable attributes. These attributes include facility information as well as distribution network application results.

3.1. DNOM Description

In order to understand the query language, it is important to first understand how data is modeled in ADMS.

The Distribution Network Object Model (DNOM) is an object-oriented object model of the distribution system network that the ADMS server loads into memory to represent the real-time connectivity of the power network. The ADMS server loads this model from the station model files (*.mod files), and persists the dynamic state of the power network to dynamic files (*.sav files).

While this memory model is really a graph, with arbitrary object relations, the model is actually persisted in a tree structure for the sake of simplicity. The root of the DNOM tree is the substation. All other queryable objects are in collections that belong to a substation. Each substation contains a collection of switches, a collection of transformers, a collection of feeders, etc.

The DNOM query engine loops through the objects in this tree to find fields in specific objects specified in a DNOM query. By definition, all DNOM queries must be routed through a station, or all stations (referred to as a “global” search).

For example, the following query lists the names and IDs of all feeders for the STATIONX substation:

```
SELECT NAME, ID FROM RT.SUB(STATIONX).FDR
```

Note that the query somewhat resembles ANSI SQL in that a set of fields are selected from this object. The resultant XML maps exactly to the names of these fields, and is organized via the hierarchical structure. For this query, for instance, assuming this station has two feeders, the result appears as follows.

```
<RT>
  <SUB>
    <FDR>
      <ID>12345</ID>
      <NAME>F1</NAME>
    </FDR>
    <FDR>
      <ID>67890</ID>
      <NAME>F2</NAME>
    </FDR>
  </SUB>
</RT>
```

The fields declared in the query above, NAME and ID, have one of two sources: either the model file

field map or a query helper function declaration. All fields that are defined in the station model files have their attributes defined in the field map.

Most other attributes, including application results, are declared with a query helper function. These query helper functions commonly translate raw data to a more user-friendly format, including translating Boolean values to “True”/“False” or “Yes”/“No”. Translations are also done for enumerations and application output results, including translating voltages stored internally as per-unit values to a more easily readable value such as 120.0 volts base (the internal per-unit value in this case is 1.00).

Basic filtering capability is also provided in the DNOM query engine. For example, to query for all currently closed breakers in a station (this is similar to the filter that is done for the fault location input and results tabular displays), a query helper function named “CLOSEDBREAKER” has been written that returns either “TRUE” or “FALSE”.

For each switch that is passed into this query helper, the function first determines if the switching device is a breaker. If the device is a breaker, the function then checks if the device is currently closed. If both conditions are met (the device is a breaker and the device is closed), the function returns “TRUE”. Otherwise, the function returns “FALSE”.

To return the name and ID of all currently closed breakers for a station (named “STATIONX”), the following query can be used:

```
SELECT NAME,ID FROM RT.SUB(STATIONX).SWT WHERE CLOSEDBREAKER = TRUE
```

In the tables in section [DNOM Component Queries](#), the columns labeled Query Helper or Field Map Name provide the declaration that is needed for querying the ADMS server for a specific attribute. This declaration can be used for queries within the Historian or the ADMS client (both attribute panels and tabular displays).

The example below demonstrates the query engine’s capability to show dynamics data using the DYNs attribute in the FROM part of the query:

FROM %MODE.DYNS.%COLLECTION

- %MODE.DYNS. is the server’s mode (RT for Real Time, ST for Study, etc.)
- %COLLECTION is the specific device collection (SWT for switches, TAG for tags, etc.)

3.1.1. Adding Custom Fields in DNOM

You can extend a DNOM model to include custom information for objects. For example, including fields from GIS that are not presented in DNOM, but provide valuable information to the user. Custom fields can then be queried like common DNOM fields and used in attribute displays, grid tabular displays, etc. Each DNOM object can have multiple custom fields.

To add custom fields for a DNOM object:

1. Open a station XML file in a text editor.

2. For the required DNOM object, add CustomField elements with names and values. For example:

```
<CustomField>
  <IsString>false</IsString>
  <Name>ElectricalLife</Name>
  <Value>10000</Value>
</CustomField>
<CustomField>
  <Name>Manufacturer</Name>
  <Value>GE</Value>
</CustomField>
```

IsString is an optional property that specifies whether the field is a string (True) or numerical (False). Numerical custom fields can be used in value comparison conditions using the Where clause in query filter. If not specified, the default value (True) is applied.

Note

A name must not be the same as the name of an existing field in DNOM for that object.

When a station model with custom fields is built, the fields can be queried using the common DNOM query syntax. For example:

```
SELECT ElectricalLife, Manufacturer FROM RT.SUB(STATIONX).SWT
```

You can also query custom field names and values using the CUSTOMFIELDXNAME and CUSTOMFIELDXVALUE query fields (currently, queries for the first eight custom fields are only supported). For example:

```
SELECT CUSTOMFIELD1NAME, CUSTOMFIELD1VALUE, CUSTOMFIELD2NAME, CUSTOMFIELD2VALUE FROM
RT.SUB(STATIONX).SWT
```

Caution

Using custom fields increases the amount memory used by the server application. Custom fields must be used sparingly to avoid a decrease in performance.

3.1.2. Commissioning/Decommissioning Equipment in DNOM

You can designate some DNOM objects as equipment to add or remove at a later date. Equipment with a “future” status will exist in the model but will not be used in the electrical analysis of the model until it is placed into an “active” state. Equipment with a “future remove” status will exist in the model and be used by the electrical analysis until it is placed into a “removed” state. After that, the electrical equipment will be ignored by the system.

The converter allows for grouping equipment that will be commissioned or decommissioned into projects that can have different stages. At the time of actual implementation in the field, the

operator can transition the states all of the equipment in a given project and stage.

To designate DNOM objects with one of these states (FUTURE or FUTUREREMOVE), the objects themselves are created as they normally would be. The state designation is applied at a station level.

To add commissioning or decommissioning information for a DNOM object:

1. Open a station XML file in a text editor.
2. Multiple projects can be defined within a given station, each with a unique ID. Stages can be created within the project. It is expected that most projects will have a default stage of 1 and that the operators rolling out the equipment will modify the actual stages based on other criteria.
3. The DNOM object with specific state is referenced by ID in the given project and stage area. For example:

```
<Project>
  <Id>CANTERBURY_Project_196018186</Id>
  <Stages>
    <Stage>
      <Id>1</Id>
      <Components>
        <Component>
          <Id>196018186</Id>
          <Type>SWITCH</Type>
          <Future>ABC</Future>
        </Component>
        <Component>
          <Id>196478570</Id>
          <Type>LINE</Type>
          <Future>ABCN</Future>
        </Component>
      </Components>
    </Stage>
  </Stages>
</Project>
```

The component state (future or future remove) is set by indicating the phases for that component that are in the given state.

Note

The Converter will validate that the component ID provided exists in the station file definition. It will also validate that the phases (either FUTURE or FUTUREREMOVE) are valid phases for the object.

3.2. DNOM Query Basics

A DNOM query has the following format:

SELECT FIELD1, FIELD2... **FROM** OBJECTCOLLECTION **WHERE** FIELD [= |< |>] VALUE [, FIELD [= |< |>] VALUE ,...]

The following items are part of the query:

- FIELDS are the fields of the network elements taken from object collections.

For the sake of simplicity, fields in this query can be considered as columns in ANSI SQL. The field names are listed in the Query Helper column or in the Field Map Name column of the tables in section [DNOM Component Queries](#). Field names are separated by a comma (",").

- The SELECT clause specifies the object fields to be received in a query result.
- "OBJECTCOLLECTION" is a collection of network objects.

The root of the object collection is a substation. "SUB(XXX)" is a specific substation named "XXX". Therefore, to get a collection of switches in a specific substation, the FROM clause is written as follows:

FROM SUB(SubstationX).SWT

where *SUB(SubstationX)* is a specific substation and *SWT* is the actual name of a switch collection.

In a DNOM query, the comma stands for a logical "AND". DNOM queries support only the logical AND operation.

To query for the names and IDs of all switching devices in a "station" that are both a closed breaker and are currently closed, the entire switch query is written as follows:

SELECT NAME,ID **FROM** RT.SUB(stationX).SWT **WHERE** CLOSEDBREAKER = TRUE, CURROPEN = 0

An XML file represents the query results (in this example, there are five breakers):

```
<RT>
  <SUB>
    <SWT>
      <ID>234234234</ID>
      <NAME>SwitchXXX</NAME>
    </SWT>
    <SWT>
      <ID>434343434</ID>
      <NAME>SwitchYYY</NAME>
    </SWT>
    <SWT>
      <ID>4342323123</ID>
      <NAME>SwitchYYY</NAME>
    </SWT>
    <SWT>
      <ID>4343234278</ID>
      <NAME>SwitchZZZ</NAME>
```

```

        </SWT>
        <SWT>
            <ID>479879834</ID>
            <NAME>SwitchAAA</NAME>
        </SWT>
        <SWT>
    </SUB>
</RT>

```

Collections in the FROM clause are always collections contained within a substation. By definition, all DNOM queries must be routed through one station, or through all stations (referred to as a “global search”, and written as “SUB(*)”).

The WHERE clause supports five logical operations:

- “=” equal to
- “!=” not equal to
- “>” more than
- “<” less than
- “,” logical AND

The WHERE clause supports logical operations between query results by using the CHECK_FOR_=1 parameter (an abbreviation from Check Field On Right). If this parameter is not used, field names are recognized only on the left side of a comparison, and the right side of a comparison is recognized as a plain value.

Note

To use values with spaces in the Where clause, enclose the values in quotes (“”).

To query for the names, IDs, nominal station names, and energizing substation names of all switching devices in the system that have different nominal and energizing substations and feeders, the entire switch query is written as follows:

```

SELECT NAME,ID,STATIONNAME,DYNSTATIONNAME, DYNFEEDERNAME, DYNFEEDERNAME2 FROM
RT.SUB(*).SWT WHERE
STATIONNAME!=DYNSTATIONNAME,DYNFEEDERNAME!=DYNFEEDERNAME2,CHECK_FOR_=1

```

SQL special functions (such as COUNT) are not supported in the WHERE clause. All special functions, such as CLOSEDBREAKER in the above query, are defined in helper functions. The WHERE clause can be omitted.

Note

By default, the maximum number of rows that are queried from the server is 10,000 (configured in NetServerMainConfig.xml). To override this value for a particular query, add MAX_RESULT_ROW = X to the Where clause, where X is the desired number of fields. To disable the maximum row count limit, set MAX_RESULT_ROW = -1.

3.2.1. Creating DNOM Queries Graphically

The Analyst Query dialog box provides an easy way to create DNOM queries and observe query results.

The Analyst Query Popup dialog box is shown with the following fields and controls:

- SELECT:** NAME,ID,STATIONNAME,DYNSTATIONNAME
- FROM:** RT.SUB(*).SWT
- WHERE:** STATIONNAME!=DYNSTATIONNAME,DYNFEEDERNAME!=DYNFEEDERNAME2,CHECK_FOR_=1
- Total Row Returned:** 5
- ☐ Managed Query
- ☐ Local Query
- Query** (button)
- Done** (button)

The results grid displays the following data:

	NAME	ID	STATIONNAME	DYNSTATIONNAME
▶	53773_Tie	133392927XTIE	COHO	RAVEN
	SW1	2F7C9518-B173-...	EQSI	SPYGLASS
	SW2	D3B456A3-C508-...	EQSI	SPYGLASS
	53774_On	133392923XON	RAVEN	COHO
*				

Figure 3. Analyst Query Pop-Up

To create a DNOM query using this feature:

1. Run the following command in the client:
NETVIEW://RT/POPUP/QUERY
2. Enter the Select, From, and Where fields as necessary.
3. Click Query.

The results grid is populated with query results. Each query field is provided in a separate column.

If the Local Query check box is set, only the client side is queried.

4. When finished, click Done.

3.2.2. Querying the List of DNOM Objects and Fields

You can for query the server for all DNOM objects and fields using the Analyst Query dialog box.

The SELECT clause supports the following values:

- **FIELDS:** Returns all query fields for the specified query object. The FROM clause must be

specified in the format RT.SUB.<ObjectName> (for example, RT.SUB.SWT). The output includes query fields separated by spaces. For example:

```
ABNDATETIME ABNDATETIME_TICKS ABNORMALSTATUS ACTUALWA ACTUALWB ACTUALWC ADDRESS...
```

- **FIELDSX:** Returns all query fields for the specified query object. The FROM clause must be specified in the format RT.SUB.<ObjectName> (for example, RT.SUB.SWT). The output includes the query object, query fields, and query field types in XML format. For example:

```
<SWT><ABNDATETIME t="s" h="1"/><ABNDATETIME_TICKS t="s" h="1"...
```

Where:

- **t:** Field type, such as string (s) or integer (i).
- **h:** Shows "1" for query fields.
- **OBJECTS:** Returns all query objects of the specified type. The FROM clause must be specified as RT.SUB or RT.OUT. The output includes query objects separated by spaces. For example:

```
ABNRML ALLENERGYSOURCE ALLTF AMSGRP AREANM AREAPF BRANCH BRNCHSHUNT  
BRSSWT...
```

- **OBJECTSX:** Returns all query objects of the specified type. The FROM clause must be specified as RT.SUB or RT.OUT. The output includes the query object, query fields, and query field types in XML format. For example:

```
<SUB><ABNRML><ABNDATETIME t="s" h="1"/><ABNDATETIMEUPDATE t="s" h="1"/>...
```

The WHERE clause supports the following values:

- **NO_VIRTUAL:** If set to 1, virtual objects and their fields are excluded from the output. The default value is 0.
- **FN_TAG:** If set to 1, field names are provided as tags for FIELDSX and OBJECTSX queries. The default value is 1.

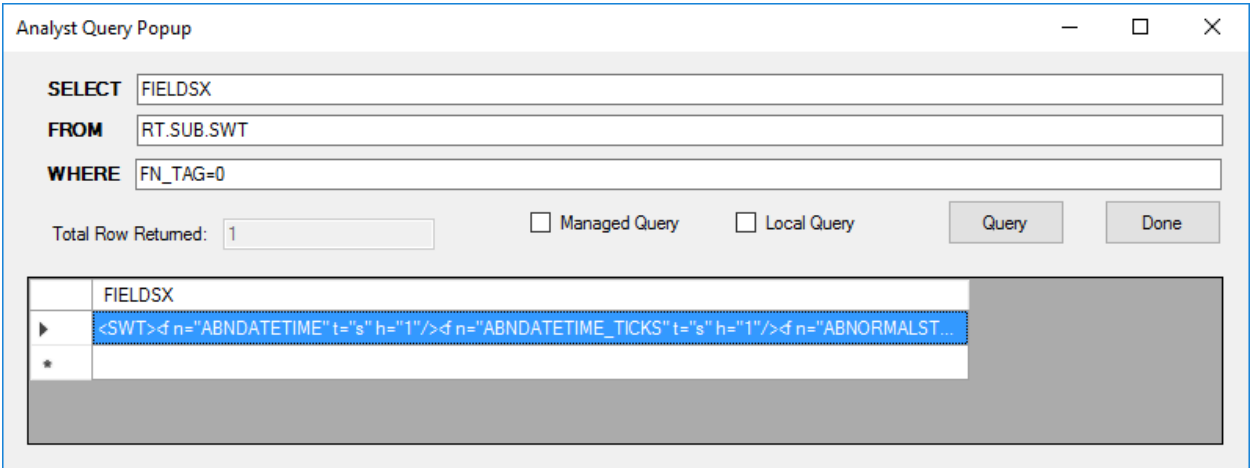


Figure 4. Querying the List of DNOM Objects and Fields

You can create local queries for all DNOM objects and fields using the Analyst DNOM Schema XML Query dialog box:

1. Run the following command in the client:

NETVIEW://RT/POPOP/SCHEMAQUERY

The Analyst DNOM Schema XML Query dialog box appears. By default, it shows all DNOM components, global objects, and virtual objects below the substation (SUB) object.

Analyst DNOM Schema Xml Query

SELECT: DNOM Objects ☒ Include Helper Fields

FROM: SUB Separated By: Space Close Save

ABNRM ALLENERGYSOURCE ALLTF AMSGRP AREANM AREAPF BRANCH BRNCHSHUNT BRSSWT BSLBPL BUS BUSANDREGULATOR BUSPLE CABNET CBLBPL CBPLE CNDMOD COMP COMPSW CONTROLDEV CSLBPL CSTMOD CSWMOD CSWPLE CUT CUTPLE DAYTYP DBANK DBLBPL DBPLE DEENRG DISTTF DMSBRANCH DMSCOMP DMSSHUNT DNADIAGNOSTIC DSOGRP DTFFLE DTLBPL ENERGYPLE ENERGYRES ENERGYRESLBPL ENERGYRESMOD ENGZSOURCE ENUTAB EXTBRC FDLBPL FDOPL FDR FILBPL FIPLE FISRCARESUMMARY FLFLTL FLRESC FLTIND FLTRIGGER FLWLIM FUTURETOP GEN GENMOD GENPLE GEOM GNDPLE GNLBPL GRLBPL GRND GSPLACEMENT INJBIF INLINEBR JUNCBR JUNCCT JUNCT LDBUS LDCAT LDLBPL LDLIM LDPLE LELPLE LKAHRS LN LNLBPL LNMOD LNPLE LNSERIES LOAD MEAS MEASILITE MEASILRES METERMEAS MHLBPL MHOLE MHPLE NDPFST NODE OBJFNT OCPROT OPTCONTROLMOVE OPTCONTROLPLAN OPTCONTROLPLANVIOLATION OPTCONTROLSOL OPTDEVICE OPTFLDLD OSMQUERYDEVICE PLANFARESULT PLANMEASISLANDRESULT PROTSET PROTSOURCE RCLBPL RCPL REGCONTROLDEF REGMODE REGULATORGROUP RESERVEDCAPACITY RFBISF RFBISFPLE SBLBPL SBPLE SCADALNK SCCSCC SCHED SCHPNT SEASON SELBPL SELECTBR SENSOR SERIES SERMOD SERPLE SHIMP SHLBPL SHPBK SHPMOD SHPPLE SLPLE SPLBOX SRVCP STINUTINT STATINT STATIONPEDIGREEFILE STLBPL STPOPL SUB SUBPLE SV SVCMOD SVCPLE SVECTR SVLBPL SWLBPL SWMOD SWPOPL SWSECT SWT SWTERM SWTPLE TAG TAPCHR TAPMOD TBLBPL TEMPGROUND TEMPOBJ TFBNK TFBPLE TFLBPL TFMOD TFPLE TGRPLE TMPSTW TOPOCOMP TSWPLE UICLCL UICNTP UICNTR UVLT LV VAULT VLEPLE VOLTLM VOLT LV VTLBPL VTPLE

2. Specify the Select and From fields.

The output text box refreshes with the new query results.

Analyst DNOM Schema Xml Query

SELECT: DNOM Object Fields ☒ Include Helper Fields

FROM: SUB of SWT Separated By: Space Close Save

ABNDATETIME ABNDATETIME_TICKS ABNORMALSTATUS ACTUALWA ACTUALWB ACTUALWC ADDRESS AFRSWCONTROLENABLED ALTGEOMETRY ALTID AMPA AMPAANGLE AMPANDANGLEA AMPANDANGLEB AMPANDANGLEC AMPANDANGLEN AMPANDANGLENEUT AMPB AMPBANGLE AMPC AMPCANGLE AMPG AMPGANGLE AREAIND AVGKVAR3PH AVGW3PH AVGVOLTA AVGVOLTB AVGVOLTC BOUNDARY BOUNDARYDEVICEISCONTROLLED BOUNDSDWDYNFDR1 BOUNDSDWDYNFDR2 BOUNDSDWFRNODE1 BOUNDSDWFRNODE2 BOUNDSDWFRNODEDYNFDR1 BOUNDSDWFRNODEDYNFDR2 BOUNDSDWSUB1 BOUNDSDWSUB2 BOUNDSDWTONODE1 BOUNDSDWTONODE2 BOUNDSDWTONODEDYNFDR1 BOUNDSDWTONODEDYNFDR2 BRAKERHASMANUALENTRY BRANCHDEENERGIZED BRANCHENERGIZED BRANCHFLOWOVERLIMITPERCENT BRANCHKVA BRANCHKVAABC BRANCHKVAB BRANCHKVAC BRANCHOPERKV BRANCHPA BRANCHPAABS BRANCHPB BRANCHPBABS BRANCHPC BRANCHPCABS BRANCHPFACTORFROMA BRANCHPFACTORFROMABC BRANCHPFACTORFROMB BRANCHPFACTORFROMC BRANCHPFACTORTOA BRANCHPFACTORTOABC BRANCHPFACTORTOB BRANCHPFACTORTOC BRANCHPHASEAENERGIZED BRANCHPHASEBENERGIZED BRANCHPHASECENERGIZED BRANCHQA BRANCHQAABS BRANCHQARELATIVE BRANCHQB BRANCHQBABS BRANCHQBRELATIVE BRANCHQC BRANCHQCABS BRANCHQCARELATIVE BRANCHREMTOP BRKFAULTINDSETSTATUSPHASE BRKGRTRPAMP BRKPHTRPAMP BRKRLCSC BYPASSWT CALCULATEFLOW CHGNOPEN CLOSEDBREAKER COMPCONFIRMCLOSED COMFLOWACTVALUE COMFLOWCAPAMARGINPERCENT COMFLOWCAPAVALUE COMFLOWDERATEDLIMITVALUE COMFLOWLIMITNAME COMFLOWLIMITPERCENT COMFLOWLIMITVALUE COMFLOWLIMITVALUEAMPS COMFLOWVALUE COMFLOWVALUEAMPS COMPHASCAPAMARGINVIOLATION COMPHASFLOWVIOLATION COMPHASMULTTAG COMPHASRESCAPIMPACT

3. Use the following controls as necessary:

- **Include Helper Fields:** For substation object queries, this option specifies whether virtual objects are shown. For object field queries, this option specifies whether query helper fields are shown.

- **Separated By:** Specifies the field separator (space, comma, or tab).
- **Save:** Opens the Save As dialog box to save the query results in a text file.

3.3. Virtual Object Queries

As stated in section [DNOM Query Basics](#), the “OBJECTCOLLECTION” in a query statement is a collection of network objects, like all line objects (LN) in one station. Sometimes, it is necessary to query from multiple objects that belong to the same category (for example, to query some fields from all branch devices). Usually, the branch devices in one substation include lines, switching devices, transformers, reactors, and cuts. In reality, there is no real device called “BRANCH” in one substation. To satisfy the query requirement in the above case, the concept of “Virtual Objects” is supported in the DMS query helper functions. The virtual objects map to multiple real DNOM objects that share some common attributes. In the above case, the virtual object is called “BRANCH”, which maps to three real DNOM objects: SWT, CUT, and LN. One can write the query statement using the following format:

```
SELECT NAME,ID FROM RT.SUB(stationX).BRANCH
```

When the query statement is executed against one virtual object, it is executed against all mapped real DNOM objects.

If the virtual object “BRANCH” has one field called “X”, you can also execute the following statement to query the field “X” from the actual “LN” object:

```
SELECT X FROM RT.SUB(stationX).LN
```

For a list of available virtual object records and their mapped real DNOM object records, refer to the “Query Objects” and “Query Object Mapping” tables in the *ADMS Modeling Reference Tables*.

3.4. DNOM Group Queries

DNOM group queries include queries that are common to certain groups of DNOM components, such as shunt objects (for example, loads), non-electrical objects (for example, fault indicators), and topology objects (for example, switches). The DNMOBJ record applies to all DNOM components and includes queries that return name, ID, and coordinates.

For a list of available DNOM group records and their mapped real DNOM object records, refer to the “Query Objects” and “Query Object Mapping” tables in the *ADMS Modeling Reference Tables*.

3.5. DNOM Component Queries

DNOM component queries include input (from station model files) and output data, and the associated query helper functions. Queries can be used in places where DNOM attributes are displayed including toolbars, grid tabular displays, and pop-up dialog boxes.

For a list of available device records, refer to the “Query Objects” table in the *ADMS Modeling Reference Tables*.

The most common DNOM component record types (names) are described in the following sections.

3.5.1. Substation

3.5.1.1. Input Attributes

The record name for the substation class is SUB.

The substation class contains the collections of all devices (lines, switches, transformers, others), measurements, and device models (such as conductor models, construction models, switch models, others) contained within a nominal station (nominal station boundaries are defined by the currently loaded station model file).

3.5.1.2. Output Attributes

The substation output class contains summary application results, such as the highest and lowest load voltages, along with the latest application solution statuses and solution times.

3.5.2. Feeder

3.5.2.1. Input Attributes

The record name for the feeder class is FDR.

The feeder class is a referenced object in ADMS. All topology objects (nodes, lines, switching devices, transformers, and other devices) reference a nominal feeder (determined at model build time) and a dynamic or currently energizing feeder. Unlike the substation class, the feeder class does not contain any collections of devices.

3.5.2.2. Output Attributes

The feeder output data includes summary power flow information such as the high-load and low-load voltages and losses.

3.5.2.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual

objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
FDR	OSMQUERYDEVICE

3.5.3. Switch

3.5.3.1. Input Attributes

The record name for the switch class is SWT.

This class includes all types of two-state switching devices, including disconnect switches, fuses, breakers, reclosers, sectionalizers, elbows, and permanent jumpers. Multiple-state devices such as composite switches (all throwovers, RAL/GRAL, and AB switches) use this class to represent the individual switches (switch links), but the overall switching behavior is not modeled within the switch class.

3.5.3.2. Output Attributes

The switch outputs include power flow information and PRV data (for breakers, reclosers, and fuses).

3.5.3.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
SWT	ABNRML
	BRANCH
	BRNCHSHUNT
	COMP
	DMSBRANCH
	DMSCOMP
	FLWLIM
	FUTURETOP
	OSMQUERYDEVICE
	SELECTBR

Actual Object Name	Mapped Virtual Object Name
	STATINPUTINT
	STATINT
	TOPOCOMP

3.5.4. Line

3.5.4.1. Input Attributes

The record name for the line class is LN.

The line class models all overhead lines and underground cables.

Based on the associated construction model and individual conductor models, a line model class is created by the converter and loaded into the server through the station model file. The associated line model contains the series impedance and shunt admittance matrices.

3.5.4.2. Output Attributes

The output class includes power flow results and estimated breaker fault currents calculated by the FL application (assumes the fault is at the end of the line).

3.5.4.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
LN	BRANCH
	BRNCHSHUNT
	COMP
	DMSBRANCH
	DMSCOMP
	FLWLIM
	FUTURETOP
	LN SERIES
	OSMQUERYDEVICE
	STATINT

Actual Object Name	Mapped Virtual Object Name
	TOPOCOMP

3.5.5. Transformer

3.5.5.1. Input Attributes

The record name for the transformer class is TF.

The transformer class includes all station power transformers (including load tap changers), auto-transformers, and regulators. Distribution (secondary) transformers are not modeled using this class.

3.5.5.2. Output Attributes

The output class includes power flow results, including voltages and flows.

3.5.5.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
TF	ALLTF
	BRNCHSHUNT
	BUSANDREGULATOR
	DMSBRANCH
	DMSCOMP
	FLWLIM
	FUTURETOP
	OPTDEVICE
	OSMQUERYDEVICE
	STATINPUTINT
	STATINT
	TOPOCOMP

3.5.6. Distribution Transformer

3.5.6.1. Input Attributes

The record name for the distribution transformer class is DISTTF.

The distribution transformer class includes only secondary transformers (non-regulating devices). These transformers can be single-phase or three-phase units. A distribution transformer can only have primary and secondary windings (no tertiary winding).

3.5.6.2. Output Attributes

The output class includes power flow results, including voltages and flows.

3.5.6.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
DISTTF	ABNRML
	ALLTF
	BRNCHSHUNT
	DMSBRANCH
	DMSCOMP
	FLWLIM
	FUTURETOP
	OSMQUERYDEVICE
	TOPOCOMP

3.5.7. Load

3.5.7.1. Input Attributes

The record name for the load class is LOAD.

The load class includes single-phase or three-phase loads connected to the load side of a distribution transformer or the primary distribution network.

3.5.7.2. Output Attributes

The output class includes power flow results, including voltages and flows.

3.5.7.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
LOAD	BRNCHSHUNT
	COMP
	DEENRG
	DMSCOMP
	DMSSHUNT
	FUTURETOP
	LDBUS
	LDLIM
	OSMQUERYDEVICE
	TOPOCOMP
	VOLTLIM

3.5.8. Shunt Impedance

3.5.8.1. Input Attributes

The record name for the shunt impedance (capacitor) class is SHIMP.

3.5.8.2. Output Attributes

The output class includes power flow results, including voltages and flows.

3.5.8.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects

that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
SHIMP	DMSCOMP
	DMSSHUNT
	FUTURETOP
	OPTDEVICE
	OSMQUERYDEVICE
	STATINPUTINT
	STATINT
	TOPOCOMP

3.5.9. Series Reactor

3.5.9.1. Input Attributes

The record name for the series reactor class is SERIES.

The reactor class includes single-phase or three-phase series reactors.

3.5.9.2. Output Attributes

The output class includes power flow results.

3.5.9.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
SERIES	BRNCHSHUNT
	DMSBRANCH
	DMSCOMP
	FLWLIM
	FUTURETOP
	LN SERIES
	OSMQUERYDEVICE
	STATINT

Actual Object Name	Mapped Virtual Object Name
	TOPOCOMP

3.5.10. Generator

3.5.10.1. Input Attributes

The record name for the generator class is GEN.

The generator class represents three-phase generators, which may be a slack source (voltage and angle targets are held) or a PV type (power and voltage targets are held). These generators can be small distributed resources or co-generation facilities.

3.5.10.2. Output Attributes

The output class includes power flow results (mainly flow injections).

3.5.10.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
GEN	ALLENERGYSOURCE
	BRNCHSHUNT
	DMSCOMP
	DMSSHUNT
	ENGZSOURCE
	TOPOCOMP

3.5.11. Energy Resource

3.5.11.1. Input Attributes

The record name for the energy resource class is ENERGYRES.

The energy resource class represents distributed energy resource objects, such as solar, wind, or battery storage. It could be single phase, two phase, or three phase.

3.5.11.2. Output Attributes

The output class includes power flow results (these could be flow injections under islanding conditions).

3.5.11.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
ENERGYRES	ALLENERGYSOURCE
	BRNCHSHUNT
	DMSCOMP
	DMSSHUNT
	TOPOCOMP

3.5.12. Ground

3.5.12.1. Input Attributes

The record name for the ground class is GRND.

The ground class represents external grounds that are not explicitly modeled on other topology objects. The ground object can be used to create a grounding network if one is required.

3.5.12.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
GRND	DMSCOMP
	DMSSHUNT
	OSMQUERYDEVICE
	TOPOCOMP

3.5.13. Switchable Sections

For more information about switchable sections, refer to the section “Switchable Sections” in the *Distribution Management System User Guide*.

3.5.13.1. Input Attributes

The record name for the ground class is SWSECT.

The switchable section class represents a nominal switchable section. The nominal switchable sections and all input parameters are determined within the converter (model build time) and are never changed in the online ADMS.

3.5.13.2. Output Attributes

The output class includes the DPF calculated net section flows (net flow = total flow in – total flow out). The flow results may not be correct if the switchable section is in an off-nominal state, such as abnormal energization or insertion of temporary modifications (cuts and jumpers).

3.5.14. Cut

3.5.14.1. Input Attributes

The record name for the ground class is CUT.

The cut class represents a cut, which is a temporary modification. Cuts can only be entered into ADMS through the user interface (a cut never exists in the station model input file).

3.5.14.2. Output Attributes

The output class includes power flow results.

3.5.14.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
CUT	ABNRML

Actual Object Name	Mapped Virtual Object Name
	BRANCH
	DMSBRANCH
	DMSCOMP
	OSMQUERYDEVICE
	TEMPOBJ
	TOPOCOMP

3.5.15. Temporary Jumper

3.5.15.1. Input Attributes

The record name for the temporary jumper class is TMPSWT.

The temporary switch class represents a temporary jumper, which is a temporary modification. Temporary jumpers can only be entered into ADMS through the user interface (a temporary jumper never exists in the station model input file).

3.5.15.2. Output Attributes

The output class includes power flow results.

3.5.15.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
TMPSWT	ABNRML
	BRNCHSHUNT
	DMSBRANCH
	DMSCOMP
	FLWLIM
	OSMQUERYDEVICE
	SELECTBR
	TEMPOBJ
	TOPOCOMP

3.5.16. Composite Switch

3.5.16.1. Input Attributes

The record name for the composite switch class is COMPSW.

The composite switch class represents multi-state switching devices. A composite switch contains 2–4 two-terminal switching devices (links). The current state of the composite switch is represented by the open/close status of the individual switch links. Currently, the composite switch class covers all throwover devices (regular and Powell ESCO), RALs/GRALs, and AB switches. The associated composite switch model contains a truth table that describes the allowed device states.

In the composite switch model, the SCADA value is integer value from SCADA for determining the device state. If the SCADA value equals to -1, the composite switch is in manual state (for example, ground), any SCADA commands will have no effect on composite switch state in DMS. If the SCADA value is not equals to -1, the composite switch is SCADA-telemetered, and the measurement change in SCADA system will update the composite switch state in DMS.

3.5.16.2. Output Attributes

The output class includes the DPF calculated current flows and voltages.

3.5.16.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
COMPSW	ABNRML
	OSMQUERYDEVICE

3.5.17. Bus

3.5.17.1. Input Attributes

The record name for the bus class is BUS.

The bus class is primarily used to represent bus work in the geographic user interface. It can be used to add voltage limits to different nodes in the network (for the limit monitor function).

3.5.17.2. Output Attributes

The output class includes the DPF calculated current voltages.

3.5.17.3. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
BUS	BUSANDREGULATOR
	FUTURETOP
	LDBUS
	OSMQUERYDEVICE
	VOLTLIM

3.5.18. Transformer Bank

3.5.18.1. Input Attributes

The record name for the transformer bank class is TFBNK.

The transformer bank class is used to group single-phase transformers that are externally connected to form wye or delta two- or three-phase transformer connections that can serve three-phase loads.

3.5.18.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
TFBNK	OSMQUERYDEVICE

3.5.19. Time Overcurrent Relay

3.5.19.1. Input Attributes

The record name for the time overcurrent relay class is RELAY.

The overcurrent relay class stores the phase trip and ground trip settings for the breakers.

3.5.20. Fault Indicator

3.5.20.1. Input Attributes

The record name for the time fault detector (indicator) class is FLTIND.

The fault indicator class represents both telemetered and non-telemetered fault indicators.

3.5.20.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
FLTIND	OSMQUERYDEVICE

3.5.21. Sensor Location

3.5.21.1. Input Attributes

The record name for the time line sensor (sensor location) class is SENSOR.

3.5.22. Vault

3.5.22.1. Input Attributes

The record name for the vault class is VAULT.

The vault class stores a collection of contained objects such as switching devices and transformers. Vaults are drawn on the geographic user interface.

3.5.22.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
VAULT	OSMQUERYDEVICE
	NONELECTRICAL

3.5.23. Switch Cabinet

3.5.23.1. Input Attributes

The record name for the switch cabinet class is CABNET.

The switch cabinet class stores a collection of contained objects such as switching devices and transformers. Switch cabinets are drawn on the geographic user interface.

3.5.23.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
CABNET	OSMQUERYDEVICE
	NONELECTRICAL

3.5.24. Duct Bank

3.5.24.1. Input Attributes

The record name for the duct bank class is DBANK.

The duct bank class stores a collection of contained lines.

3.5.24.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
DBANK	OSMQUERYDEVICE
	NONELECTRICAL

3.5.25. Manhole

3.5.25.1. Input Attributes

The record name for the manhole class is MHOLE.

The manhole class stores a collection of contained objects such as switching devices and transformers.

3.5.25.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes (called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
MHOLE	NONELECTRICAL

3.5.26. Splice Box

3.5.26.1. Input Attributes

The record name for the splice box class is SPLBOX.

The splice box class stores a collection of contained objects such as switching devices and lines.

3.5.26.2. Virtual Attributes

Besides the attributes described in “Input Attributes” and “Output Attributes”, more attributes

(called “Virtual Attributes”) can be queried against this object if this object is mapped to some virtual objects as described in section [Virtual Object Queries](#). The following table lists the virtual objects that are mapped to this object.

Actual Object Name	Mapped Virtual Object Name
SPLBOX	OSMQUERYDEVICE
	NONELECTRICAL

3.5.27. Trouble Ticket

3.5.27.1. Input Attributes

The record name for the trouble ticket class is TTK.

The trouble ticket class represents all trouble tickets placed.

3.5.28. Tags

The record name for the tag class is TAG. “DYNs” contains a tag collection under the name “TAGS”. The attribute “DYNs” stands for dynamics data. Using RT.DYNs.TAGS lists of all tags for the RT mode.

The tag class represents all tags placed (EMS and DMS).

3.6. Global Object Queries

The global file input data queries are listed in this section.

For a list of available global object query records, refer to the “Query Objects” table in the *ADMS Modeling Reference Tables*.

3.6.1. Construction Model

The record name for the construction model class is CSTMOD.

The construction model class contains the spatial coordinates for the conductor framing/construction model (coordinates are also provided for underground cables).

3.6.2. Conductor Model

The record name for the conductor model class is CNDMOD.

The conductor model class contains the impedance data and operational limit data for the various conductor types.

3.6.3. Load Category

The record name for the load category class is LDCAT.

The load category model class includes load voltage limit data and the associated family of load shapes that are used in the load allocation application.

3.6.4. Schedule

The record name for the schedule class is SCHED.

The schedule class describes the schedule, including the X and Y axis units, the shape, and special parameters needed to load schedules (including an associated cold load schedule).

The time-based schedules support sub-hours. For a given time, the value in the Y-axis will be calculated using the interpolation method. If you want the value in the Y-axis to only be based by hour, define the "shape" as 2 (step, not linear).

3.6.5. Schedule Point

The record name for the schedule point class is SCHPNT.

The schedule point class contains the values stored in the schedule. A schedule point can contain one or two values on the Y axis.

3.6.6. Season

The record name for the season class is SEASON.

The season class contains the start and end dates for different seasons (a season does not need to correspond to Summer, Fall, Winter, or Spring).

3.6.7. Day Type

The record name for the day type class is DAYTYP.

The day type class describes both regular day types (such as weekday, weekend) and specific calendar days (such as holidays).

3.6.8. Transformer Model

The record name for the transformer model class is TFMOD.

The transformer model class provides impedance data and operational limits. This class may also provide the association to the GE transformer type library, which is needed for the network applications.

3.6.9. Tap Changer Model

The record name for the tap changer model class is TAP.

The tap changer model class provides tap changer details, including the maximum and minimum tap, and tap increments.

3.6.10. Switch Model

The record name for the switch model class is SWMOD.

The switch model class provides switch characteristics (ganged operation, load break capability) and operation limits.

3.6.11. Shunt Impedance Model

The record name for the shunt impedance model class is SHPMOD.

The shunt impedance model class provides operational limits and loss data.

3.6.12. Series Reactor Model

The record name for the series reactor model class is SERMOD.

The series reactor model class provides impedance data and operational limits.

3.6.13. Generator Model

The record name for the generator model is GENMOD.

The generator model class provides impedance data and generator output limits.

3.6.14. Energy Resource Model

The record name for the energy resource model is ENERGYRESMOD.

The energy resource model class provides impedance data and energy resource object output limits.

3.6.15. Energy Resource Program Definition

The record name for the energy resource program definition class is ENERGYRESPROGRAMDEF.

The energy resource program definition class represents the characteristics of an energy resource program to include name, program name, program (enrollment) number, and program type.

3.6.16. Composite Switch Model

The record name for the composite switch model class is CSWMOD.

3.6.17. Optimization Station Group

The record name for the optimization station group class is DSOGRP.

3.6.18. Enumeration Table

The record name for the Enumeration table class is ENUTAB.

4. In-Memory Data Storage

ADMS supports in-memory data tables that can be populated from the server and exposed to clients via the standard query interface. These tables can be used by server plugins, such as LVM System Monitor, LVM System Monitor Read-Only, or Orchestrator (for look-ahead DPF solutions), to store data that is unknown to the server and is not part of the DNOM model.

The data stores (table names and fields) are defined in the code. The client can query data stores using the standard query process, similar to querying DNOM objects. For more details, see section [DNOM Query Basics](#). A single query can only access data from DNOM or from a data store.

The following are example queries:

```
SELECT NAME,FORECASTPERIOD FROM RT.DATASTORE.LSMSYSTEM
```

```
SELECT NAME,FORECASTPERIOD FROM RT.DATASTORE.LSMSYSTEMRO
```

```
SELECT MAXREALROC FROM RT.DATASTORE.LSMAREA
```

```
SELECT MAXREALROC FROM RT.DATASTORE.LSMAREARO
```

```
SELECT INTERVALMINUTES FROM RT.DATASTORE.QSTSJOBSUMMARY
```

5. Viewing Station Model Data

The DNOM Viewer tool opens station model files to reveal the station model data.

To open DNOM Viewer from the Windows Start menu, navigate to **E-terra** > Distribution > Tool > Modeling Tools > DNOM Viewer.

The Summary tab in DNOM Viewer lists the available DNOM Records. The Message tab provides the activity log.

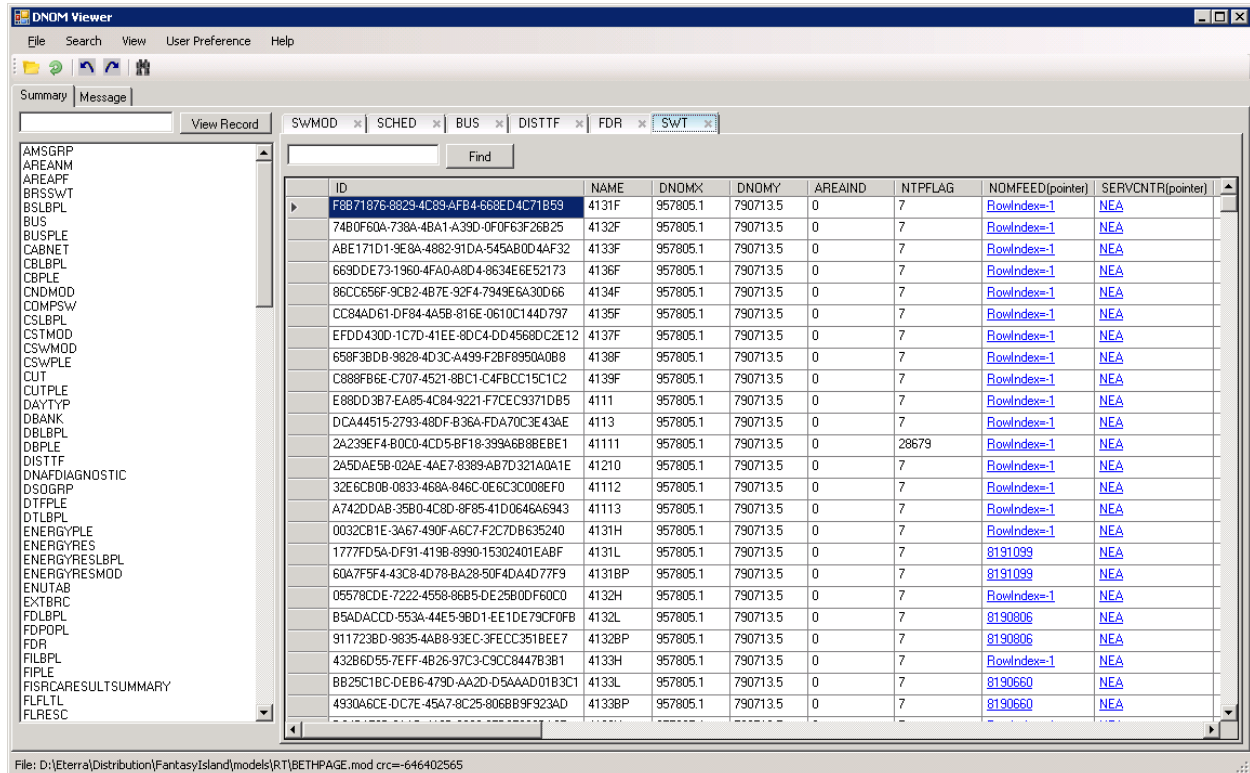


Figure 5. DNOM Viewer Query Pop-Up Window

The DNOM Viewer controls include the following:

- **Open:** Shows the Open dialog box to open a station model file.
- **Reload:** Reloads the currently open station model file.
- **Exit:** Closes the application.
- **Find:** Opens the Find dialog box to find values in the DNOM model.

Find

User Preference

Records

- All
- AMSGRP
- AREANM
- AREAPF
- BRSSWT
- BSLBPL
- BUS
- BUSPLE

Fields

- All

Find what: 8813

☐ Match Exact Case

☐ Match Whole Word Only

Find Close

	Object Name	Field Name	Value
▶	LN	NAME	17881350
	LN	NAME	22468813
	LN	DNOMY	818813.9
	SWT	NAME	8813
	SWT	DNOMY	808813.3
	NODE	ID	16888130
	NODE	NAME	16888130
	NODE	DNOMY	808813.3
	LOAD	ID	16888135XSCI
	MEAS	ID	RAVEN.SW.881...
	MEAS	NAME	RAVEN.SW.881...
	TFBNK	ID	16888135
	LNPLE	AV	937058.3937240

- **Go Back:** Navigates to the previous record. Available after clicking a pointer to another DNOM record.
- **Go Forward:** Advances to the next record.
- **Clear User Preference:** Clears (removes) user preferences.